

SPEND MANAGEMENT AI HACKATHON · 2026

Verified Regeneration

The product memory and governance layer between human intent and AI execution — so every AI agent working on Spend Management knows what the product is, what must stay true, and who decided it.

Swarn Singh · Live demo on the Spend Management estate · 7,800+ knowledge nodes

THE PROBLEM

AI made writing code fast. Understanding still starts over — every session.

Every AI session begins at zero

Before touching Spend Management, an engineer or AI agent must rediscover what exists, what it means, what depends on it, and why it was built that way. That work is redone for every session, every engineer, every model.

Confident ≠ correct

A coding agent will implement “let users edit card controls in read-only mode” fluently and quickly — it has no way to know a person already decided read-only must stay read-only.

In a bank, that’s not a productivity gap — it’s a control gap

Product decisions live in people’s heads and old tickets. When AI regenerates code faster than humans can review it, undocumented intent is the first casualty — and the compliance function becomes the bottleneck.

AI accelerates execution.

Understanding — and governance — start over.

WHAT VR IS

Copilot, but it remembers your product — and remembers who decided what.

- **Product graph:** 7,800+ nodes connecting capabilities → behavior → implementation → evidence → tests → integrations → human decisions, for the real Spend Management estate.
- **Works inside Copilot:** engineers ask normal questions; VR supplies bounded, cited context and validates every claim mechanically.
- **Governance built in:** human decisions are first-class records with a named reviewer and an append-only audit history.

**A plain coding agent reconstructs the product.
VR remembers it.**

Every fact is labeled by how it is known

- EXISTS TODAY** backed by direct code evidence
- INFERRED** supported interpretation, not reviewed
- RULE TO PROTECT** a named person approved it
- UNKNOWN** VR says so — it never guesses

The core principle: confidence is never authority

No model output — however confident — becomes a product rule. Only an explicit human decision does. And once decided, no AI can silently undo it.

HOW IT WORKS

Four steps, and only one of them needs a person

1 · Learn the product

VR reads the estate once and builds a memory: capabilities, behavior, evidence, and what it could not determine



2 · Answer with citations

Every question answered from that memory — with file and line references, and honest unknowns



3 · Draft the rules

One command turns a capability into a reviewable draft of what it does today. Nothing invented



4 · A person decides

Only a named human turns behavior into a rule. This is the step AI is not allowed to take

The AI proposes. VR checks.

Copilot does the reasoning; VR supplies the evidence and rejects any claim that cites nothing. A confident sentence with no source never reaches the answer.

It says when it doesn't know

“Should we enhance MCC from 23 to 43?” → VR asks what 23 → 43 means, and stops. In a bank, a confident wrong answer costs more than no answer.

It runs where the code is

Inside VS Code, on the Copilot licenses engineers already have. No code, data, or decisions leave the machine.

Ask it something it should refuse to answer

► Should we enhance our MCC from 23 to 43 as part of a NextGenMCC effort?

■ I need one detail before I analyze this.

What does 23 → 43 represent in NextGenMCC?

...and it stops. No impact analysis. No risks.
No confident invention.

Why start here

Every one of us has watched an AI answer this kind of question beautifully and wrongly. VR found the MCC areas in the product — it could easily have produced an impressive analysis. It refuses, because it cannot tell what 23 → 43 means.

The rule underneath

When meaning is missing, ask — never manufacture. That single behavior is what makes the rest of the answers worth trusting.

Watch a rule get made — then watch it defend itself

- ▶ Wrap the Card Controls capability
 - ⚠ DRAFT FROM EXISTING BEHAVIOR — NOT APPROVED INTENT
 - ...14 behaviors, each cited to a file and line...
 - ▶ Promote the analytics-event line as a rule
 - ✓ Recorded — approved by Swarn Singh, 09:41
 - ▶ ...forty seconds later...
 - ▶ Remove the analytics event from the card-add spend-limit step to reduce noise.
- This change touches a rule a person already approved.
Keep the rule · Change the rule · Decide later · Cancel

The moment

That rule did not exist two minutes ago. You watched it get made, and you watched it stop a plausible request. In six months, when whoever made it has moved teams, it still will.

Not an AI opinion

VR is not blocking because a model thinks this is risky. It is blocking because a named person decided it — and it can show you who, and when.

Copilot does the thinking. VR checks its work.

Inside VS Code, on your Copilot licence

Ask “should I remove Adobe Analytics?” and Copilot names the risks — using evidence VR retrieved and bounded, not a web search and not its memory.

Then every claim is checked

8 risks supported by evidence. 0 rejected as unsupported. A risk that cites nothing is thrown away before you ever see it — including risks the model was confident about.

The question it asked back

“Is Adobe Analytics the provider behind this code, or does ‘analytics’ also cover the SpendAnalytics feature that must remain?” — the distinction a careful engineer would raise.

```
RISK Analytics definitions removed while still  
referenced by event-driven web code  
→ src/analytics/events/cardControls.ts
```

```
RISK Cross-repository deployment gap  
→ touched code spans four repositories
```

```
RISK Existing test suite breakage  
→ 20 test files cover the touched area
```

Every line above carries a file and line number.
Anything that did not, VR removed.

WHY IT MATTERS HERE

Bank controls, expressed as engineering artifacts

CONTROL	WHAT VR PRODUCES CONTINUOUSLY
Change management	Governed change packages; conflicting changes stop for a human decision
Segregation of duties	Models propose, validators gate, only named humans approve rules
Audit trail	Append-only decision history; per-answer evidence bundles with citations
Institutional memory	Product knowledge that survives attrition, reorgs, and model changes

Operational value now

Faster, safer engineering on Spend Management: cross-repo answers with citations in seconds, honest unknowns, and rules that can't be lost in Slack threads.

Strategic value next

This is the on-ramp to Verified Regeneration as a discipline: wrap legacy behavior → govern constraints → hand any AI a contract it must satisfy — the path to safely regenerating implementations.

HOW A TEAM ADOPTS IT

Five commands. Cited answers on day one.

1	<code>vr init</code>	VR reads your repos — read-only — and maps files, tests and languages. Nothing is inferred or assumed yet.	~10 minutes
2	<code>vr semantic scan</code>	Builds the product graph: capabilities, behaviors, evidence, unknowns. AI proposes in bounded packages; VR validates every claim against citations.	unattended
3	<code>vr explore · ask in Copilot</code>	Your team asks normal questions and gets cited answers, the rules that apply, and honest unknowns — across every repo at once.	day one
4	<code>vr wrap "<capability>"</code>	Drafts a reviewable manifest of what a capability actually does today — every line cited and labeled, nothing invented.	minutes
5	<code>vr promote <line> --reviewer</code>	A person turns observed behavior into a rule to protect. From that moment it is enforced on every future change, by any AI.	seconds

AI proposes · VR validates · **only a named person approves** — runs entirely inside the bank's environment, on the Copilot licenses engineers already have. No code, data, or decisions leave the machine.

What's real today — and what we won't claim yet

Working now, on the real estate

- Persistent product graph · 7,800+ nodes, integrity-verified
- Conversational Ask/Analyze inside Copilot, with citations
- Product identity & context layer (what the product *is*)
- Wrap → Promote → Constraint pipeline with audit history
- Approved-rule gate on change requests + governed packages
- Per-answer evidence bundles · fail-closed clarification
- 53 automated tests · runtime integrity manifest

Deliberately not claimed

- Autonomous implementation or “safe to merge” — packages are handed to agents; independent verification is the roadmap
- Signed per-build evidence bundles, shadow verification, drift detection — designed, not yet built
- Cold-start quality on an arbitrary estate — exactly what a pilot measures

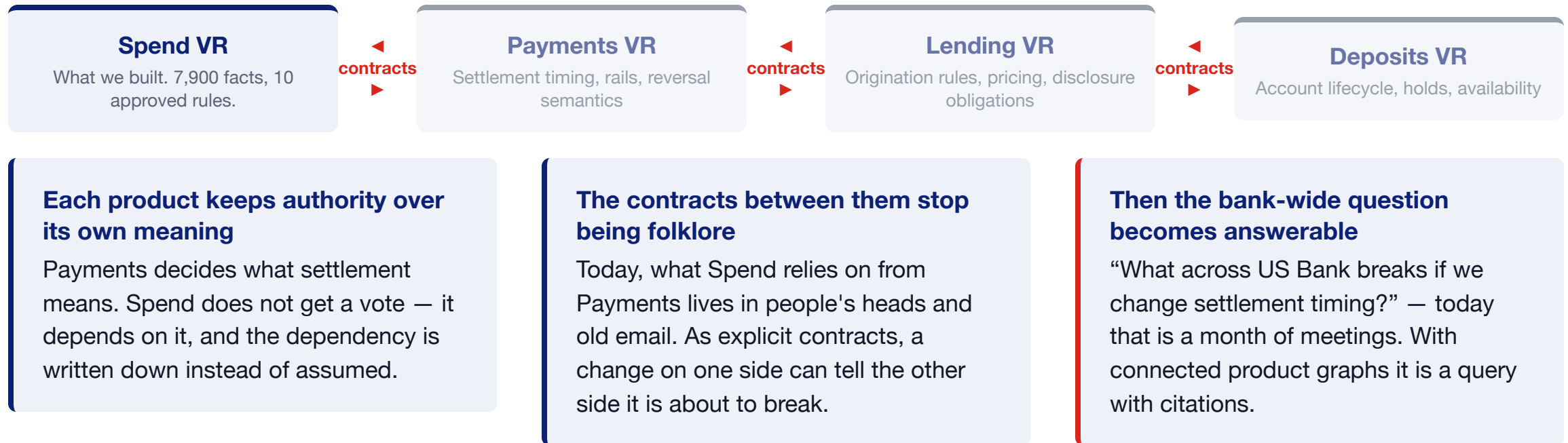
Why we say this out loud

In a regulated environment, credibility is won by what you decline to promise. VR is built on that principle — it even applies it to its own answers.

WHY THIS IS BIGGER THAN ONE PRODUCT

Spend Management is the first estate, not the only one

Nothing in this is about card controls. It is about giving AI a durable memory of what a product means — and every US Bank product has the same gap.



WHAT CHANGES ON MONDAY

The same layer, felt differently by each seat

Engineers

Today: rediscover the product every session; find the landmine after the incident.

Ask a question, get an answer with file and line. See the rules that constrain the change *before* writing code — and hand any AI agent a change with its obligations attached.

Product managers

Today: decisions live in a document nobody reads, and drift silently.

Ask what the product actually does today versus what we believe. Record a decision once and have it enforced on every future change, by every person and every model.

Business & risk

Today: evidence is assembled by hand, the week before the audit.

See every rule, who approved it, and when. Change approval and audit trail become artifacts the pipeline produces continuously.

AI will write more of our software every quarter.

The question is whether it will understand what our products mean — or keep asking us to remember for it.